The Brilliant Employee · 05

My router chose the same model 5,000 times

The first outside test of the part of my system that picks which AI model does each job. With exploration off it sent all 5,000 rows to one model. A coin flip has two outcomes.

sgnk.ai/brilliant ↗

Sagnik Mitra

Why this exists

A language model writes a large share of my production code. Wrong work reads exactly like right work, and no prompt fixes that. So the system below exists to find out when the answers were wrong.

From one week, none of it visible from inside the tool: a sizing step ignored on 796 of 816 tasks, and two fixes written up as done that were never made.

THE SYSTEM, AS OF 7 AUGUST 2026, 14:51Z	
rules written	11 June, 57 days ago
ledger running	27 June, 41 days
tasks logged	3,103, across 3,620 rows
controls on tool calls	**6 registered, 4 that can refuse**

You can rent the model. You cannot rent the part that tells you it was wrong.

Five parts, and where today sits

Five parts sit around the model. Each post takes one of them.

sizing check	sizes a job before a model is picked
model picker	recommends which model gets the job
safety stops	refuse the irreversible, before it runs
grader	grades finished work pass or fail
task log	at least one append-only line per task

Listed in the order they act on a task, not the order they were built.

Today is about the model picker, marked above.

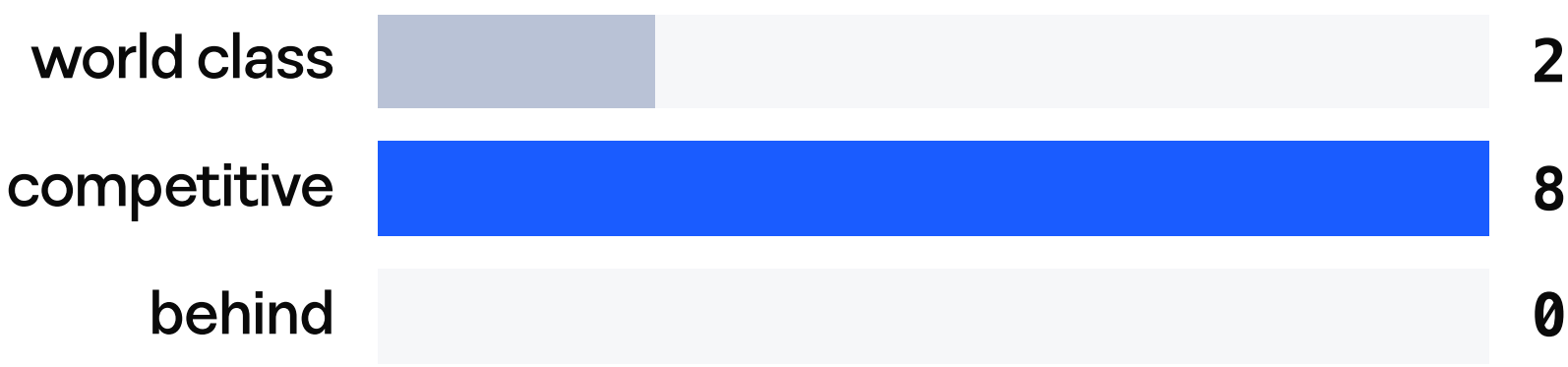
> **ASIDE.** Only the safety stops can refuse. The other four observe: one advises, one recommends, one grades after the fact, one records.

This page is the map. Nothing here needs another post to make sense.

Every grade came from a test I wrote

My automation system has a component I call the model picker, the part that recommends which model gets each job. Cheap model for routine work, expensive model when the job earns it.

For its first month, every grade the model picker received came from inside my own workspace. The loudest was a 44 agent self-review campaign across the whole system, and not one of those graders was built by someone else.



How the 44 agent campaign graded the system across 10 dimensions. It never once found the system behind.

> **ASIDE.** That campaign was honest work. It measured coverage, whether a component exists for each concern. It could not measure ranking.

A grader that shares your assumptions inherits your blind spots.

On day 30 it took a test I did not write

> **Plain words: ROUTERBENCH.** A public benchmark of 401,467 precomputed inference outcomes, 36,497 rows scored against each of 11 models. You replay a routing policy over it and it scores the quality and the cost of the assignments the policy makes. Nobody who built it has ever seen my system.

Replayed with exploration off, meaning the policy never tries a model it has not already scored, my shipped configuration pulled the cheap arm on all 5,000 rows and scored 0.3108, exactly the always-cheap score. For scale, across the full 36,497 rows a random assignment scores 0.5202 and always-cheap scores 0.3061. Different sample sizes, so read those as context, not as a ranking.

> **ASIDE.** Date of the exam: 2026-07-27 UTC, day 30 of the system's life. The first external measurement it ever had.

The exam was not written for coding agents and its model pool predates the two tiers I route between. That is the honest caveat on the score, and it is why the lesson here is about who writes your exam.

The first replay from outside the building found one choice, repeated.

Both verdicts are honest, one is blind

Both columns are honest. They measure different things. The internal campaign measured coverage: does machinery exist for each concern. The exam measured behavior: do the assignments vary at all.

INTERNAL, SELF-RUN	EXTERNAL, ROUTERBENCH
campaign **44 agents, mine**	live config, 5,000 row replay **0.3108**
world class **2 of 10**	random baseline, full run **0.5202**
competitive **8 of 10**	always-cheap, full run **0.3061**
behind **0 of 10**	graders built by **strangers**

Coverage was fine. Behavior never varied. Nothing in the left column was positioned to notice.

> **ASIDE.** The audit that followed put it plainly: that verdict is evidence about coverage, not an independent ranking, and no external benchmark had ever been run against the system.

A component existing is not the same as that component still deciding.

I misread the exam on the first pass

My first report said we passed at 0.7699. The number was real and the verdict was wrong. I had added 5 percent forced exploration to the simulation to keep it from sticking, then forgot to account for it. What it measured was a policy that wanders. The shipped one stands still.

FIRST REPORT	CORRECTED
score **0.7699**	score **0.3108**
verdict **a policy that wanders**	verdict **a policy that stands still**
hidden input **5% forced exploration**	hidden input **none**

> **ASIDE.** Both numbers sit in the same written report. The corrected one is in bold.

The correction is the part worth keeping. A number that is real and a verdict that is wrong is the ordinary shape of a self-graded result, and it took an instrument I did not write to separate them.

Even the outside exam got marked by my own test harness on the first pass.

A safety rule locked it onto one model

Plain words: THE ABSTENTION GATE. A rule that refuses to trust a model's score until that model has 5 recorded results. Sensible on its own. With exploration off, only the model picked first ever collects those 5.

1. Task zero: no model has the 5 observations the abstention gate requires
2. The policy falls back to the default arm, the cheap model
3. Only that arm accumulates observations, so only it can ever pass the gate
4. Every other model is scored minus infinity, forever
5. Result: 5,000 simulated tasks, one distinct model chosen 5,000 times

ASIDE. On its own the abstention gate is a good rule.

A zero exploration policy does not route. It repeats.

I had been tuning the wrong knob

A refutation is only depressing until you spend it. Retuning the model picker, the part that recommends the model, against the exam showed my pessimism setting was a red herring and the cost term was the real lever.

THE RETUNE, EXPLORATION ON, SCORED AGAINST ALWAYS-EXPENSIVE

cost weight 0, the shipped value, exploration on	90.0% quality at 76.1% cost
cost weight 100	**77.0% quality at 7.4% cost**
cost weight 1000	75.7% quality at 6.5% cost

One sweep of the cost term, every row with exploration on. The middle row is the recommended design point. The live rule still carries no cost term at all.

Exploration was armed 72 minutes later, in a separate commit under an unrelated subject line, carrying the minimum detectable effect calculation the arming rule had always required and nobody had done. The next verdict arrives with a sample size instead of a feeling.

External exams do not just grade the system. They tune it.

A test you wrote cannot surprise you

Every check I write shares my assumptions: my task mix, my definition of success, my simulation code, my blind spots. An instrument built by strangers disagrees with you in ways you cannot anticipate. That is not a flaw of the instrument. That is the product.

A TEST I WRITE	A TEST I DID NOT WRITE
task mix **mine**	built by **strangers**
definition of success **mine**	has seen my system **never**
simulation code **mine**	disagrees with me **in ways I cannot anticipate**
blind spots **shared with mine**	run against me so far **1, this one**

The audit written the day before the exam said flatly that no external benchmark had ever been run against this system. This was the first. No second one has run since. One exam in, the score stands at one refuted internal verdict, and every comparative claim I make about this system is self-assessed until a second instrument reports.

Your own tests share your assumptions.
A benchmark you did not write does not.

One model, 5,000 times. That is repeating, not routing.

The derivation, the misread I made on the first pass, and the proof that settled it.

sgnk.ai/brilliant ↗

Sagnik Mitra

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

`sgnk.ai · in/sagnikmitra · github.com/sagnikmitra`